THE UNIVERSITY OF LAHORE

FACULTY OF INFORMATION TECHNOLOGY

ARTIFICIAL INTELLIGENCE — ASSIGNMENT 2

Customer Churn Prediction Web Application Using Flask

Based on Assignment 1: Customer Churn Prediction Using Machine Learning

Student Name	Hamayal
Roll Number / SAP ID	70172424
Class / Section	D
Instructor	Sir Ansar Naseem
Submission Date	18th - October - 2026

Dataset: WA_Fn-UseC_-Telco-Customer-Churn.xlsx
Required deployment platform: PythonAnywhere

Table of Contents

1. Abstract

2. Introduction and Problem Statement

3. Objectives and Assignment Requirements

4. Dataset and Model Carried Forward

5. Tools and Technologies

6. System Architecture and Workflow

7. Project Structure

8. Frontend Design and Input Features

9. Flask Backend and Prediction Flow

10. Model Saving and Loading

11. Local Testing and Observed Results

12. Input Validation and Error Handling

13. PythonAnywhere Deployment

14. Limitations and Future Improvements

15. Conclusion

Appendix A: Evidence Screenshots

Appendix B: Final Submission Checklist

References

Report status: The local Flask application and result page were observed running. PythonAnywhere deployment and final online testing have not yet been evidenced and are marked as pending.

1. Abstract

This project converts a machine-learning customer churn model into a browser-based web application. The interface collects telecom customer information and sends it to a Python Flask backend. The backend loads a saved Logistic Regression model, processes the submitted values, and displays a readable prediction indicating whether the customer is likely to stay or churn, together with an estimated churn probability. The local application was run successfully at http://127.0.0.1:5000 and a result page was observed. The assignment also requires deployment to PythonAnywhere; that online deployment remains to be completed and verified before final submission.

2. Introduction and Problem Statement

Customer churn occurs when a customer stops using a company's service. Churn can affect revenue and customer growth. A predictive model can help a telecom provider identify customers who may be at risk and prioritize retention efforts. Assignment 1 developed and evaluated machine-learning classifiers. Assignment 2 focuses on making the selected model usable through a web interface and deploying it online.

Problem statement: Provide a simple website where a user enters customer attributes and receives a clear, model-generated churn prediction without needing to execute notebook code manually.

3. Objectives and Assignment Requirements

- Carry forward the best-performing model from Assignment 1.

- Save the trained model and any feature/preprocessing information required for inference.
- Build a clear Flask project with templates and static CSS.
- Create an HTML form with suitable labels and input controls for customer features.
- Connect form submissions to Flask and apply training-compatible preprocessing.
- Display an understandable Churn/No Churn result.
- Handle missing, invalid, or unexpected inputs with user-friendly messages.
- Test multiple customers locally, including a likely churn case and a likely no-churn case.
- Deploy the application on PythonAnywhere and test its public URL from another browser/device.
- Submit the complete project, model/preprocessing artifacts, PDF report with evidence, and working live URL.

4. Dataset and Model Carried Forward

The project is based on the Telco Customer Churn dataset named WA_Fn-UseC_-Telco-Customer-Churn.xlsx. The Assignment 1 report describes 7,043 customer records and 21 original columns. The target column is Churn, where Yes represents churn and No represents a customer who stayed. The reported preprocessing removed customerID, converted TotalCharges to numeric, converted the target to 1/0, and applied one-hot encoding to categorical data.

The Assignment 1 report records Logistic Regression as the selected model because it outperformed the tested Random Forest model on the reported test split. The reported Logistic Regression metrics were Accuracy 80.70%, Precision 66.14%, Recall 55.88%, and F1-score 60.58%. These are Assignment 1 experiment results, not a fresh evaluation of the web application.

Metric	Logistic Regression	Random Forest
Accuracy	80.70%	78.50%
Precision	66.14%	61.79%
Recall	55.88%	49.73%
F1-score	60.58%	55.11%

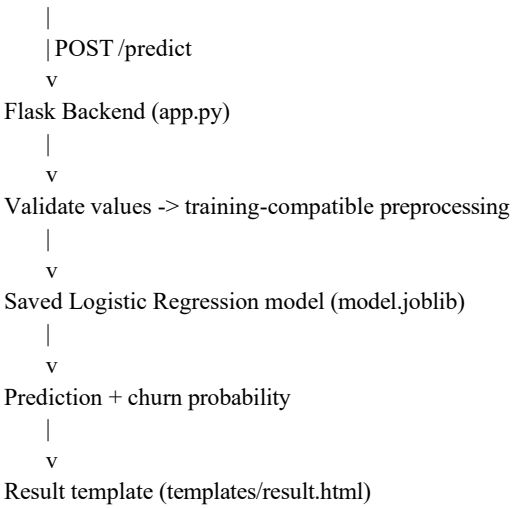
Important integration requirement: the web app must use the same feature order and encoding as model training. Creating dummy columns from a single submitted row can produce incorrect encodings if categories are not represented consistently. The application should retain the original training feature names and align new input to those columns, or use a saved preprocessing pipeline.

5. Tools and Technologies

Technology	Purpose
Python	Backend programming and model inference
Flask	Routes, form submission handling, and rendering pages
HTML	Customer input form and result page structure
CSS	Layout, spacing, colors, and readability
pandas	Preparing customer input and aligning features
scikit-learn	Classification model and churn probability
joblib	Saving and loading the trained model
PythonAnywhere	Required online hosting platform

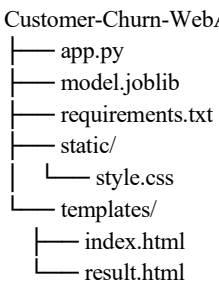
6. System Architecture and Workflow

Customer (Browser)
    |
    v
HTML Form (templates/index.html)



The form sends customer details to the Flask prediction route. Flask should validate the values, convert them into the exact feature representation expected by the model, call predict() and predict_proba(), and render a readable result. The result page should not expose raw arrays or internal model objects.

7. Project Structure



The observed VS Code project view contains the backend file, saved model, requirements file, templates folder, and static CSS folder. This matches the core structure specified by the assignment.

8. Frontend Design and Input Features

The form uses labelled fields and dropdowns for categorical values and numeric fields for tenure and charges. The observed interface includes Gender, Senior Citizen, Partner, Dependents, Tenure (months), Phone Service, Multiple Lines, Internet Service, Online Security, Online Backup, Device Protection, Tech Support, Streaming TV, Streaming Movies, Contract, Paperless Billing, Payment Method, Monthly Charges, and Total Charges.

The design is intended to make the form straightforward for a normal user. Labels clarify each feature, dropdowns limit categorical inputs to expected values, and the Predict Churn button submits data to the backend.

9. Flask Backend and Prediction Flow

The backend file app.py starts a Flask development server, loads model.joblib, displays the form on the home route, and processes form submissions on /predict. The result page displays the predicted class and estimated probability. The model file is loaded with joblib, and pandas is used to prepare customer data for prediction.

Typical inference steps (conceptual):

1. Read submitted form values.
2. Validate required fields and numeric ranges.
3. Convert values to the same columns/encoding used during training.
4. Align input columns with the saved training feature order.

5. prediction = model.predict(customer_features)
6. probability = model.predict_proba(customer_features)[0][1]
7. Render result.html with a user-friendly message.

The exact implementation should be checked against app.py before submission. The essential rule is that web-time preprocessing must match Assignment 1 preprocessing.

10. Model Saving and Loading

The trained model was exported from the notebook using joblib and downloaded into the local project. The downloaded file was confirmed to be 2,079 bytes, and the Flask app subsequently started and loaded it.

```
import joblib
joblib.dump(best_model, "model.joblib")
```

```
# In app.py
model = joblib.load("model.joblib")
```

During local startup, scikit-learn displayed an InconsistentVersionWarning: the model was saved with version 1.6.1, while the local environment reports version 1.9.1. The app started, but the versions should be aligned for dependable submission. A warning-free load and prediction test should be captured after resolving the environment version.

11. Local Testing and Observed Results

The Flask server started locally at http://127.0.0.1:5000. The browser displayed the customer form and, after submission, a separate prediction result page. In one observed run the page showed “Customer is likely to STAY” and an estimated churn probability. A second submission also returned a result. This confirms that the local form-to-result route works at a basic level.

The assignment requires at least two contrasting test cases: one expected churn case and one expected no-churn case, and web results should be compared with predictions from the same model using correctly encoded features. This comparison has not yet been documented.

12. Input Validation and Error Handling

The assignment requires empty, incorrect, or unexpected values not to crash the application. Before final submission, test:

- Submit the form with a required field missing; show a helpful message rather than a server error.
- Enter invalid text into numeric fields such as tenure, Monthly Charges, or Total Charges.
- Check that tenure is within a sensible range (0–72 months) and charges are non-negative.
- Check that dropdown values are limited to the options expected by the model.
- Confirm the user can return to the form after viewing the result.

Do not claim these cases passed until they have actually been tested.

13. PythonAnywhere Deployment — Pending

The assignment requires the website to be accessible online from another device or browser without depending on the local computer. The evidence currently available shows local execution only. Therefore, PythonAnywhere deployment and final online testing are pending.

1. Create or sign in to a PythonAnywhere account.
2. Upload the complete project, including app.py, templates, static CSS, model.joblib, and requirements.txt.
3. Install compatible dependencies in the PythonAnywhere virtual environment.
4. Configure the Web tab and WSGI file to point to the Flask application.
5. Set the working directory and model file path correctly, then reload the web app.

6. Open the public URL and test form submission, prediction, styling, and error handling.
7. Insert the final live URL and screenshots of PythonAnywhere configuration and the deployed website.

Live URL: TO BE ADDED AFTER DEPLOYMENT — do not submit a placeholder as a working URL.

14. Limitations and Future Improvements

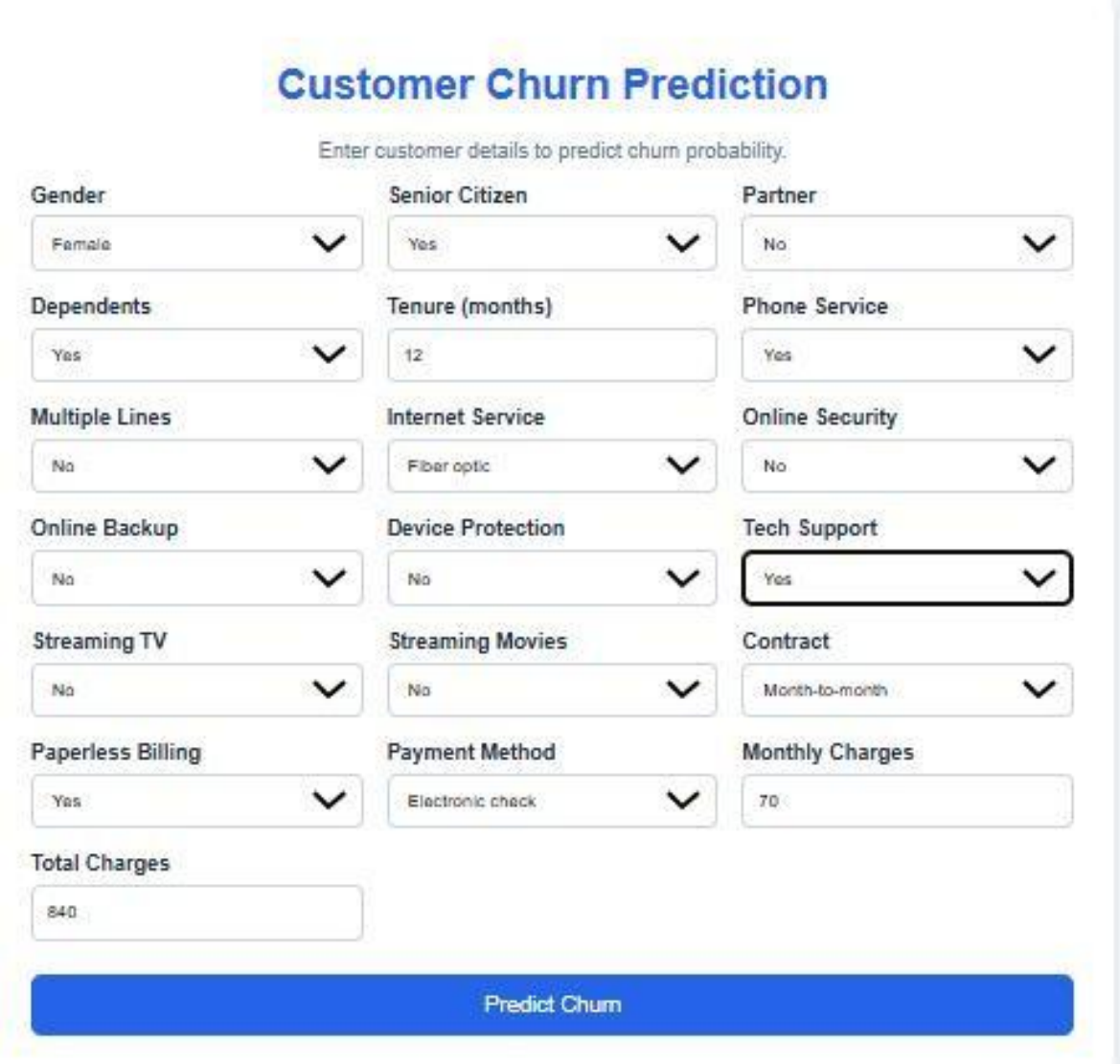
- The local environment showed a scikit-learn version mismatch warning.
- Prediction reliability depends on using the same encoding and feature order as model training.
- The observed local result proves the route returns output, but does not by itself prove prediction correctness.
- The model estimate is a risk score, not a guarantee of future behavior.
- Future improvements include a saved preprocessing pipeline, stronger validation, additional test cases, and monitoring.

15. Conclusion

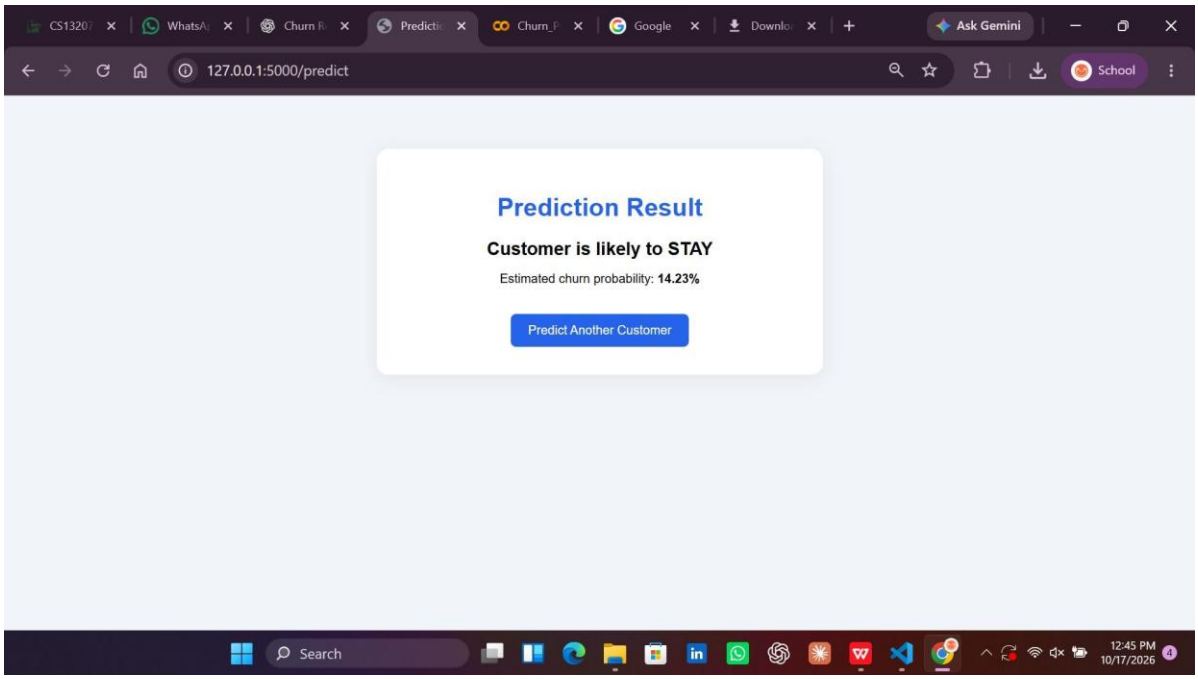
A basic Flask-based Customer Churn Prediction web application has been created and run locally. The customer form accepts customer details, and the result page displays a predicted stay/churn outcome with an estimated churn probability. The project has the core folders and files required by the assignment. Before final submission, preprocessing and feature alignment must be verified, the scikit-learn version mismatch should be resolved, input validation and contrasting test cases must be tested, and the application must be deployed and tested on PythonAnywhere.

Appendix A: Evidence Screenshots

Figure A1. User-provided screenshot of the customer input form and local result page/project folder. This evidence shows the visible interface and that the local prediction route returned a result.



Local application evidence supplied during development.



Local application evidence supplied during development.

Evidence Still Required

- Screenshot of the notebook cell saving model.joblib and the resulting file size.
- Screenshot of the complete project folder structure.
- Screenshot of important app.py code and validation/preprocessing logic.
- Screenshot showing one likely churn test and one likely no-churn test, verified against the same model.
- Screenshot of PythonAnywhere Web/WSGI configuration.
- Screenshot of the final deployed website and public URL.

Appendix B: Final Submission Checklist

☐ Complete Flask project folder or ZIP file

☐ model.joblib plus all required preprocessing/feature metadata

☐ requirements.txt with compatible package versions

☐ Final report with actual screenshots and explanations

☐ Local test for both churn and no-churn cases

☐ Validation/error-handling test evidence

☐ PythonAnywhere deployment completed and reloaded

☐ Working public URL tested from a separate browser/device

☐ Student details completed on the cover page

References

- University of Lahore, Faculty of Information Technology. Artificial Intelligence, Assignment 2: Customer Churn Prediction Web Application Using Flask (assignment brief).
- Assignment 1 project report: Customer Churn Prediction Using Machine Learning, based on the Telco Customer Churn dataset.
- Python documentation: https://docs.python.org/3/

- Flask documentation: https://flask.palletsprojects.com/
- scikit-learn model persistence documentation: https://scikit-learn.org/stable/model_persistence.html